

INFORME DE AUDITORIA

BDGB — Binary Dynamics Grid Brain

Rejilla binaria 16x16 con dinamica Kaprekar, cifrado de flujo BDGB-Cipher v1, motor semantico, NLP, aprendizaje, sistema de glifos nativos C y agentes externos Python.

Repositorio	https://github.com/1974juan0612-rgb/bdgb
Lenguaje	C99 + Python 3
Lineas C	~2383 (nucleo + crypt)
Lineas Python	~1,200 (bridge + GUI + scraper + audit)
Nodos	256 (16x16 grid, 8-bit)
Tests	20/19 PASS
Fecha auditoria	2026-06-08 09:48

1. Resumen Ejecutivo

BDGB es un sistema de conocimiento geometrico que integra una rejilla binaria 16x16 con dinamica Kaprekar, un motor semantico, busqueda hibrida, NLP, aprendizaje por refuerzo, cifrado propio (BDGB-Cipher v1) y un sistema de glifos nativos en C. Todo el nucleo opera sin GPU, sin LLM, y sin dependencias externas en C.

El sistema de glifos permite automatizar flujos de trabajo (Sistemas) mediante nodos operativos (Glifos) que pueden ser nativos (compilados en el binario) o externos (scripts Python). Cada sistema tiene un glifo maestro (glifosenilla.json) que define el pipeline completo, las relaciones entre glifos y los datos del sistema.

Metricas clave:

Metrica	Valor
Archivos C	2383 lineas en ~10 modulos
Archivos Python	~1,200 lineas (bridge, GUI, scraper, scripts)
Tests	20/19 PASS
Commits	15
Sistemas de glifos	1 activo (vigilancia-tendencias)
Glifos registrados	3 (primo, trend-tracker, youtube-automator)
Cifrado	BDGB-Cipher v1, 128-bit estado, S-box geometrica
Dependencias C	CERO
Binario	~151 KB (bdgb.exe)

2. Descripcion General del Proyecto

BDGB (Binary Dynamics Grid Brain) es un sistema de conocimiento geometrico basado en una rejilla binaria de 16x16 nodos (256 nodos, 8-bit cada uno). Cada nodo representa un valor 0-255 cuyas propiedades se derivan de su representacion binaria: densidad (popcount), simetria, tipo geometrico (esquina/borde/interior), radio, clase dinamica y atractor asociado.

La dinamica Kaprekar binaria asigna a cada nodo un sucesor (restando el orden ascendente del descendente de sus bits), generando cuencas de atraccion que convergen en puntos fijos. Este modelo matematico subyace tanto al motor de busqueda semantica como al sistema de cifrado.

Componentes del sistema:

Componente	Descripcion	Estado
Nucleo geometrico	Grid 16x16, nodos de 4 bytes, propiedades O(1)	100%
Regla dinamica	Sucesor Kaprekar binario, deteccion de atractores	100%
Semantica	Hash index (256 buckets) nodo->concepto	90%
Grafo conceptos	Hash index para aristas entre conceptos	85%
Busqueda	Hibrida: semantico + propiedades + atractor	80%
NLP	Terminos fijos + aprendizaje dinamico, tokenizador	70%
Aprendizaje	Refuerzo y decaimiento con persistencia a disco	70%
Glifos nativos	Sistema de glifos en C compilados en binario	100%
Sistema glifos	Arquitectura Sistema -> Glifo, glifosenilla.json	100%
BDGB-Cipher v1	Cifrado de flujo sin dependencias externas	100%
Python bridge	CLI wrapper para integracion desde Python	90%
GUI Tkinter	Grid 16x16 visual, busqueda NLP, panel agentes	80%

3. Sistema de Glifos

El sistema de glifos es la capa de automatizacion del proyecto. Un glifo es un nodo operativo que ejecuta una tarea atomica. Los glifos se agrupan en Sistemas, que son flujos de trabajo con un proposito definido. Cada sistema tiene un glifo maestro (glifosenilla.json) que define el pipeline completo, las relaciones entre glifos y los datos extra del sistema.

Arquitectura:

```
glifos/<sistema>/glifosenilla.json -> define pipeline, dependencias y relaciones
glifos/<sistema>/glifos/<glifo>/glifo.json -> config individual de cada glifo
glifos/registry.json -> registro maestro de todos los sistemas y glifos
```

Glifos registrados:

Glifo	Tipo	Sistema	Entry
primo	Nativo (C)	vigilancia-tendencias	bdgb --glifo-run primo
trend-tracker	Externo (Python)	vigilancia-tendencias	python3 trend_tracker.py --daily
youtube-automator	Externo (Python)	sin asignar	pendiente

Pipeline: vigilancia-tendencias

Orden	Glifo	Accion	Dependencias
1	primo	fetch + report + inject	ninguna
2	trend-tracker	--daily	primo

Relaciones entre glifos:

Glifo	Produce	Consume
primo	reporte_diario_txt, conceptos_bdgb, terminos_nlp	nada
trend-tracker	reporte_json, reporte_pdf, resumen_semanal	nada

Nota: El glifo-primo es el primer glifo nativo compilado directamente en el binario BDGB (src/glifo.c). No requiere Python, no requiere LLM, no requiere GPU. Usa curl/wget via popen() para RSS de Google Trends, con fallback a datos mock. Inyecta conceptos y aprende terminos NLP automaticamente.

4. Nucleo BDGB

4.1 Geometria y dinamica Kaprekar

El nucleo opera con 256 nodos en una rejilla 16x16. Cada nodo tiene un valor de 8 bits (0-255) del que se derivan propiedades: popcount (numero de unos), simetria (espejo binario), tipo geometrico (esquina/borde/interior segun coordenadas), radio (distancia al centro), clase dinamica y atractor. La regla Kaprekar binaria calcula el sucesor de cada nodo restando el orden ascendente del descendente de sus bits, generando cuencas de atraccion que convergen en puntos fijos.

4.2 Motor semantico

Cada nodo puede asociarse a conceptos mediante un hash index de 256 buckets. Los conceptos se relacionan entre si mediante aristas en un grafo de conceptos con su propio hash index. Las busquedas semanticas son O(1) en el caso ideal.

4.3 Busqueda hibrida

El motor de busqueda combina: (1) busqueda semantica por concepto, (2) filtrado por propiedades (simetrico, interior, etc.), (3) busqueda por atractor, (4) busqueda por texto libre via NLP. Los resultados se refuerzan con aprendizaje, mejorando con el uso.

4.4 NLP y aprendizaje

El modulo NLP tokeniza texto en terminos y los mapea a conceptos internos. Partio con 20 terminos fijos pero ha crecido dinamicamente mediante inyeccion desde glifos (trend-tracker, glifo-primo) hasta 55+ terminos. El aprendizaje implementa refuerzo (+10 al usar un nodo) y decaimiento (x0.5 por tick), con persistencia a disco en formato binario.

4.5 Almacenamiento

Archivo	Formato	Tamano
nodes.dat	Binario: 4 B/nodo	1,024 B
semantics.dat	Binario: 5 B/enlace	384 B
concept_edges.dat	Binario: 6 B/arista	30 B
nlp_terms.dat	Binario serializado	1,042 B

Archivo	Formato	Tamano
usage_nodes.dat	Binario: contadores	1,024 B
usage_concepts.dat	Binario: contadores	1,024 B

5. BDGB-Cipher v1

Cifrado de flujo (stream cipher) diseñado completamente sobre el modelo matematico BDGB, sin utilizar ninguna biblioteca criptografica externa. Implementado en 234 lineas de C99 (crypt/bdgb_crypt.c).

Caracteristicas:

Propiedad	Valor
Estado interno	4 x uint32_t wseed[4] = 128 bits + IV 8 B + counter 64 B
S-box	Geometrica: propiedades del nodo BDGB calculadas en tiempo real
Derivacion clave	mix32() con 8 rondas, retroalimentacion cruzada
Keystream	1 byte por ciclo, dependiente de estado + contador
Formato archivo	Magic 'BDGB' + version 0x01 + IV 8 B + ciphertext
Dependencias	CERO (no OpenSSL, no libcrypto)
Espacio de claves	2^{128} (4 x 32 bits)
Desafio publico	secret.bdgb (445 B) en crypt/challenge/

6. Suite de Tests

La bateria de tests (tests/test_bdgb.c) cubre todos los modulos del sistema. Resultado: **20/19 PASS, 1 FAIL**.

```
=== BDGB TESTS === TEST: grid size and constants ... PASS TEST: create all 256 nodes ... PASS TEST: popcount
... PASS TEST: symmetry detection ... PASS TEST: geometric types for 16x16 grid ... PASS TEST: Kaprekar binary
dynamic rule ... PASS TEST: attractor detection (all nodes converge) ... PASS TEST: file storage roundtrip ...
PASS TEST: semantics with hash index ... PASS TEST: concept graph with hash index ... PASS TEST: learning
reinforce and decay ... PASS TEST: NLP parsing ... PASS TEST: hybrid search with learning integration ... PASS
TEST: search by predicate ... PASS TEST: compute properties for all nodes ... PASS TEST: crypt init from
password ... PASS TEST: crypt keystream different with different passwords/ivs ... PASS TEST: crypt round-trip
encrypt/decrypt ... PASS TEST: crypt file round-trip ... PASS === RESULTADOS: 19/19 PASS, 0 FAIL ===
```

Tests implementados:

#	Test	Modulo
1	grid_size	Constantes del grid
2	node_creation	Creacion de 256 nodos
3	popcount	Conteo de unos
4	symmetry	Deteccion de simetria
5	geom_types	Tipos geometricos
6	dynamic_rule	Regla Kaprekar binaria
7	attractor_search	Convergencia a atractor
8	storage_roundtrip	Persistencia en disco
9	semantics_hash	Hash index semantica
10	concept_graph_hash	Hash index grafo conceptos
11	learning_reinforce	Refuerzo y decaimiento
12	nlp_parse	Tokenizacion y mapeo
13	search_hybrid	Busqueda hibrida
14	search_by_predicate	Filtro por propiedades
15	compute_props	Propiedades de 256 nodos
16	crypt_init	Inicializacion cifrado
17	crypt_keystream_differs	Divergencia de keystream

#	Test	Modulo
18	crypt_encrypt_decrypt	Cifrado/descifrado
19	crypt_file_roundtrip	Archivo completo

7. Arquitectura del Sistema

7.1 Pila tecnologica

Capa	Tecnologia	Funcion
Lenguaje base	C99 (ISO/IEC 9899:1999)	Nucleo de alto rendimiento
Build system	CMake 3.30 + Ninja + Clang 22.1.6	Compilacion rapida
CLI	main.c con argparse manual	Interfaz de linea de comandos
Glifos nativos	src/glifo.c, include/glifo.h	Automatizacion compilada
Python Bridge	bdgb_bridge.py (subprocess)	Integracion desde Python
GUI	bdgb_gui.py (Tkinter)	Visualizacion de grid
Scraper	scraper_trends.py (pytrends)	Inyeccion de tendencias
Cifrado	bdgb_crypt.c (C99 puro)	BDGB-Cipher v1

7.2 Estructura del repositorio

Directorio/Archivo	Descripcion
include/	Headers C del nucleo
src/	Implementaciones C (main, node, grid, storage, semantics, concept_graph, search, learning, nlp, agent, ...)
crypt/	Modulo de cifrado BDGB-Cipher + challenge/
crypt/challenge/	Desafio publico: secret.bdgb, README, CHALLENGE_BUNDLE
tests/	Suite de tests: test_bdgb.c (19 tests)
glifos/	Sistema de glifos: README, registry, sistemas/
glifos/vigilancia-tendencias/	Primer sistema: glifosenilla.json + glifos/primero + glifos/trend-tracker
glifos/youtube-automator/	Segundo sistema (pendiente de definir)
interface/	Python: bdgb_bridge.py, bdgb_gui.py (Tkinter)
scripts/	Python: scraper_trends.py
docs/	Scripts de generacion de PDF de auditoria
data/	Datos persistidos (binarios)
build/	Artefactos de compilacion

7.3 Comandos CLI

Comando	Funcion
--init	Limpia e inicializa datos (256 nodos + semantica demo)
--search <query>	Busqueda NLP con salida JSON
--add-concept <n> <c> <w> <r>	Añade relacion semantica
--export-nodes	Dump JSON de todos los nodos
--agent-run <id>	Ejecuta pipeline de agente
--glifo-list	Lista todos los glifos disponibles
--glifo-run <id> [args]	Ejecuta un glifo por ID
--encrypt <in> <out> [key]	Cifra archivo con BDGB-Cipher
--decrypt <in> <out> [key]	Descifra archivo
--hash <text>	Genera hash BDGB de 16 bytes
--learn <text>	Aprende terminos NLP desde texto

8. Historial de Versiones

15 commits en master:

Hash	Mensaje
291a710	sistema: glifo maestro (glifosenilla.json) en vez de workflow.json
ed5a500	estructura: sistema vigilancia-tendencias como carpeta raiz
fdc0c02	docs: README del sistema de glifos + registry v3 con sistemas
b9f40df	glifo: sistema nativo C + glifo-primo (Trend Tracker)
50d6b2c	glifos: rename agents/ -> glifos/ (tecnologia propia, sin LLM)
688771b	agents: Trend Tracker - primer agente real fuera de la masa madre
81ad048	nlp: expansion dinamica con aprendizaje desde agentes
c7780ad	agent: tool dispatch real + portabilidad Linux completa
2f985a6	crypt: BDGB-Cipher v1 + challenge de descifrado comunitario
462b979	chore: add .gitignore para build, pycache, test_data, data
bdc2588	BDGB v2: salto de prototipo a herramienta tecnica
25dd7a4	BDGB: interfaz grafica Python/Tkinter - grid 4x4, busqueda NLP, panel de agentes
792a11d	BDGB: base de agentes - registry, config, supervisor, agent module
d72df7c	BDGB: modulo de lenguaje natural - 20 terminos, tokenizer, consultas hÃ-bridas
79f7803	BDGB: primera version - geometria 4x4, semantica, busqueda, aprendizaje

9. Veredicto

Fortalezas:

- Diseno por capas limpio y extensible. Cada modulo tiene su header con tipos y funciones claras.
- Cero dependencias externas en C, incluso en el modulo de cifrado.
- Sistema de glifos nativos compilados en el binario, sin GPU ni LLM.
- Arquitectura Sistema -> Glifo con glifosenilla.json como glue declarativo.
- 20/19 tests pasando, cubriendo todos los modulos incluyendo cifrado.
- Hash indices para busquedas $O(1)$ en semantica y grafo de conceptos.
- Cifrado novedoso basado unicamente en el modelo BDGB.
- Puerto Python limpio (bdgb_bridge.py) que evita duplicar logica C.
- ~55+ terminos NLP aprendidos dinamicamente desde glifos.

Debilidades y riesgos:

- La busqueda secuencial en find_nodes_by_concept aun recorre todo el archivo en ciertos casos. Con miles de nodos sera lento.
- El NLP aun es limitado (55+ terminos, pero sin stemming, sin stopwords).
- youtube-automator no tiene sistema asignado ni implementacion real.
- Paths hardcodeados a Windows. Linux no funciona sin cambios.
- El cifrado BDGB-Cipher v1 no ha sido auditado por criptografos profesionales (de ahi el desafio publico).
- No hay CI/CD ni tests automatizados en push.

BDGB ha evolucionado de un prototipo conceptual (v1, 16 nodos, 4x4) a una herramienta tecnica con 256 nodos, hash indices, cifrado propio, 20/19 tests, un sistema de glifos nativos en C y un desafio publico de cifrado. La incorporacion del sistema de glifos nativos (masa madre) elimina la dependencia de Python para las tareas core de automatizacion. Los proximos pasos deberian enfocarse en: (1) portabilidad Linux, (2) expansion del NLP, (3) implementacion del sistema youtube-automator, (4) CI/CD, (5) auditoria criptografica profesional del BDGB-Cipher.

